Debounce

Run once, after the user stops typing

debounce.js

```js
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
const search = debounce(callApi, 300);
input.addEventListener("input",
  e => search(e.target.value));
```

output `6 calls` → `1 call`

The 4 ideas

01 The problem
Calling the API on every keystroke: "iphone" = 6 requests.

02 The fix
Wait until input stops for `delay` ms, then run the function once.

03 clearTimeout
Each new keystroke cancels the pending call and starts a fresh timer.

04 Closure
The returned function remembers `timer` between calls.

Step by step (typing "iphone")

1. Type `i`: a 300ms timer starts.
2. Each next letter arrives before 300ms: old timer cleared, new one starts.
3. User stops. 300ms pass: `callApi("iphone")` runs **once**.

Debounce vs Throttle

The follow-up question interviewers love

throttle.js

```js
function throttle(fn, limit) {
  let waiting = false;
  return (...args) => {
    if (waiting) return;
    fn(...args);
    waiting = true;
    setTimeout(() => (waiting = false), limit);
  };
}
```

use case	pick	why
search input	debounce	only the final text matters
autosave, resize end	debounce	run after activity stops
scroll, mousemove	throttle	steady updates while it continues
infinite scroll	throttle	check position every X ms

Traps to mention

- **Create the debounced function once.** Re-creating it on every render resets the timer, so it never fires (React: useMemo / useRef).
- **Cancel on unmount.** A pending timer can still fire after the component is gone.

Your 20-second answer

"Debounce waits until events stop for a set time. Each new event clears the old timer and starts a new one, so only the last call runs. The timer lives in a closure. Throttle instead runs at most once per interval while events keep firing."